UNIT 5 - Logistic Regression

Mitch Truong

14102025

```
# Import the necessary libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib as mpl
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.metrics import precision_score, recall_score, f1_score, roc_auc_score
```

Get the dataset from the Logistic Regressions Dataset folder

Create the Task dataset as follows:

1. Load the .csv file into a data frame in Python.

```
# Load the dataset
df = pd.read_csv('task_data.csv')
# Display the first 5 rows
df.head()
```

	person_age	person_gender	person_education	person_income	person_emp_exp	person_home_ownership	loan_amnt	loan_intent	loan_int_rate	loan_percent_income	cb_person_cred_hist_length	credit_score	previous_loan_defaults_on_file	loan_status
0	22.0	female	Master	71948.0	0	RENT	35000.0	PERSONAL	16.02	0.49	3.0	561	No	1
1	21.0	female	High School	12282.0	2	OWN	1000.0	EDUCATION	11.14	0.08	2.0	504	Yes	0
2	25.0	female	High School	12282.0	0	MORTGAGE	35000.0	MEDICAL	12.87	0.49	3.0	635	No	1
3	23.0	female	Bachelor	79753.0	0	RENT	35000.0	MEDICAL	15.23	0.44	2.0	675	No	1
4	24.0	male	Master	66135.0	1	RENT	35000.0	MEDICAL	14.27	0.53	4.0	586	No	1

2. Describe the data, provide information on the dataset

```
# Display dataset shape
print("Dataset Shape: (df.shape)")
print("Rows: {df.shape[0]}, Columns: {df.shape[1]}")
# Display dataset info
Rows: 45000, Columns: 14
```

```
# Display dataset information
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45000 entries, 0 to 44999
Data columns (total 14 columns):
 #   Column                              Non-Null Count  Dtype
---  ------                              --------------  -----
 0   person_age                          45000 non-null  float64
 1   person_gender                       45000 non-null  object
 2   person_education                    45000 non-null  object
 3   person_income                       45000 non-null  float64
 4   person_emp_exp                      45000 non-null  int64
 5   person_home_ownership               45000 non-null  object
 6   loan_amnt                           45000 non-null  float64
 7   loan_intent                         45000 non-null  object
 8   loan_int_rate                       45000 non-null  float64
 9   loan_percent_income                 45000 non-null  float64
 10  cb_person_cred_hist_length          45000 non-null  float64
 11  credit_score                        45000 non-null  int64
 12  previous_loan_defaults_on_file      45000 non-null  object
 13  loan_status                         45000 non-null  int64
dtypes: float64(8), int64(3), object(3)
memory usage: 4.8 MB
```

```
# Statistical summary of the dataset
df.describe()
```

	person_age	person_income	person_emp_exp	loan_amnt	loan_int_rate	loan_percent_income	cb_person_cred_hist_length	credit_score	loan_status
count	45000.000000	4.500000e+04	45000.000000	45000.000000	45000.000000	45000.000000	45000.000000	45000.000000	45000.000000
mean	27.764178	80.319674e+03	5.411233	9682.737778	11.006606	0.139725	5.867489	632.608756	0.222222
std	6.045108	6.044234e+04	5.963532	6314.886691	2.978808	0.087551	3.879702	50.435865	0.415744
min	20.000000	8.000000e+03	0.000000	500.000000	5.420000	0.000000	2.000000	390.000000	0.000000
25%	24.000000	4.704929e+04	1.000000	5000.000000	8.590000	0.070000	3.000000	601.000000	0.000000
50%	26.000000	6.704094e+04	4.000000	8000.000000	11.011860	0.120000	4.000000	640.000000	0.000000
75%	30.000000	9.789924e+04	8.000000	12237.250000	12.369652	0.190000	8.000000	678.000000	0.000000
max	144.000000	7.200766e+06	125.000000	35000.000000	20.000000	0.660000	30.000000	850.000000	1.000000

```
# Check for missing values
print("Missing Values:")
```

```
person_age                            0
person_gender                         0
person_education                      0
person_income                         0
person_emp_exp                        0
person_home_ownership                 0
loan_amnt                             0
loan_intent                           0
loan_int_rate                         0
loan_percent_income                   0
cb_person_cred_hist_length            0
credit_score                          0
previous_loan_defaults_on_file        0
loan_status                           0
dtype: int64
```

```
# Check for duplicate rows
print("Number of duplicate rows: (df.duplicated().sum())")
Number of duplicate rows: 0
```

3. Conduct Logistic Regression

In this scenario we are conducting logistic regression to predict loan approval status (0 = Denied, 1 = Approved).

4. Preprocess data if needed

```
# Step 1: Identify the target variable
target = 'loan_status' # 0 (denied) or 1 (approved)
# Finding how many loans were approved vs denied
print("Target variable distribution:")
print(df[target].value_counts())
# Calculate percentages
print("\n")
Target variable distribution:
loan_status
0    35000
1    10000
Name: count, dtype: int64
```

```
# Encode categorical variables
# Create a copy to avoid modifying original data
df_processed = df.copy()
# Find all categorical (text-based) columns
categorical_cols = df[df.select_dtypes(include='object')]
# Loop through each categorical column and transform it
for col in categorical_cols:
    # Create a new LabelEncoder for this specific column
    le = LabelEncoder()
    # Fit and transform the column, replacing text with numbers
    df_processed[col] = le.fit_transform(df_processed[col])
# Check what we have after encoding
print("After encoding:")
print("Shape:", df_processed.shape)
print("Data types:")
df_processed.dtypes
After encoding:
Shape: (45000, 14)
Data types:
person_age                          float64
person_gender                         int64
person_education                      int64
person_income                       float64
person_emp_exp                        int64
person_home_ownership                 int64
loan_amnt                           float64
loan_intent                           int64
loan_int_rate                       float64
loan_percent_income                 float64
cb_person_cred_hist_length          float64
credit_score                          int64
previous_loan_defaults_on_file        int64
loan_status                           int64
dtype: object
```

```
# Step 2: Separate features (X) and target (y)
X = df_processed.drop(columns=[target])  # X = all columns EXCEPT the target (inputs/values)
y = df_processed[target]  # y = only the target column (outputs/labels)
print("Features (X) shape:", X.shape)  # Shape: (number of rows, number of features)
print("Target (y) shape:", y.shape)  # Shape: (number of rows,)
Features (X) shape: (45000, 13)
Target (y) shape: (45000,)
```

5. Print the correlation coefficient matrix

```
# Compute correlation matrix
correlation = df_processed.corr()
print(correlation)
                                      person_age  person_gender  person_education  \
person_age                             1.000000       0.017518          0.010207
person_gender                          0.017518       1.000000          0.000000
person_education                       0.010207       0.000000          1.000000
person_income                          0.020796       0.000000          0.000000
person_emp_exp                         0.854113       0.010279          0.012994
person_home_ownership                  0.000000       0.000000          0.000000
loan_amnt                              0.000742       0.000017          0.000000
loan_intent                            0.000000       0.000000          0.000000
loan_int_rate                          0.011661       0.000342          0.000000
loan_percent_income                    0.020422       0.000000          0.000000
cb_person_cred_hist_length             0.852077       0.010399          0.012772
credit_score                           0.000000       0.000000          0.000000
previous_loan_defaults_on_file         0.012727       0.000763          0.001757
loan_status                            0.010375       0.000763          0.001757

                                      person_income  person_emp_exp  \
person_age                             0.020796        0.854113
person_gender                          0.000000        0.010279
person_education                       0.000000        0.012994
person_income                          1.000000        0.027037
person_emp_exp                         0.027037        1.000000
person_home_ownership                  0.000000        0.000000
loan_amnt                              0.246615        0.005748
loan_intent                            0.000000        0.000000
loan_int_rate                          -0.045414       0.011231
loan_percent_income                    -0.544981       0.020291
cb_person_cred_hist_length             0.027036        0.852326
credit_score                           0.040796        0.016351
previous_loan_defaults_on_file         -0.023474       0.010356
loan_status                            -0.544534       0.010375

                                      person_home_ownership  loan_amnt  \
person_age                                         0.000000   0.000742
person_gender                                      0.000000   0.000017
person_education                                   0.000000   0.000000
person_income                                      0.000000   0.246615
person_emp_exp                                     0.000000   0.005748
person_home_ownership                              1.000000   0.000000
loan_amnt                                          0.000000   1.000000
loan_intent                                        0.000000   0.000000
loan_int_rate                                      0.000000   0.146755
loan_percent_income                                0.000000   0.570387
cb_person_cred_hist_length                         0.000000   0.004201
credit_score                                       0.000000   0.021796
previous_loan_defaults_on_file                     0.000000   0.011656
loan_status                                        0.000000   0.146755

                                      loan_intent  loan_int_rate  \
person_age                               0.000000       0.011661
person_gender                            0.000000       0.000342
person_education                         0.000000       0.000000
person_income                            0.000000      -0.045414
person_emp_exp                           0.000000       0.011231
person_home_ownership                    0.000000       0.000000
loan_amnt                                0.000000       0.146755
loan_intent                              1.000000       0.000000
loan_int_rate                            0.000000       1.000000
loan_percent_income                      0.000000       0.125687
credit_score                             0.000000      -0.042636
previous_loan_defaults_on_file           0.000000       0.023202
loan_status                              0.000000       0.125687

                                      loan_percent_income  \
person_age                                       0.020422
person_gender                                    0.000000
person_education                                 0.000000
person_income                                   -0.544981
person_emp_exp                                   0.020291
person_home_ownership                            0.000000
loan_amnt                                        0.570387
loan_intent                                      0.000000
loan_int_rate                                    0.125687
loan_percent_income                              1.000000
cb_person_cred_hist_length                       0.032650
credit_score                                     0.039390
previous_loan_defaults_on_file                   0.048789
loan_status                                      1.000000

                                      cb_person_cred_hist_length  \
person_age                                            0.852077
person_gender                                         0.010399
person_education                                      0.012772
person_income                                         0.027036
person_emp_exp                                        0.852326
person_home_ownership                                 0.000000
loan_amnt                                             0.004201
loan_intent                                           0.000000
loan_int_rate                                         0.000000
loan_percent_income                                   0.032650
cb_person_cred_hist_length                            1.000000
credit_score                                          0.016243
previous_loan_defaults_on_file                        0.011656
loan_status                                           0.000000

                                      credit_score  \
person_age                                0.000000
person_gender                             0.000000
person_education                          0.000000
person_income                             0.040796
person_emp_exp                            0.016351
person_home_ownership                     0.000000
loan_amnt                                 0.021796
loan_intent                               0.000000
loan_int_rate                            -0.042636
loan_percent_income                       0.039390
cb_person_cred_hist_length                0.016243
credit_score                              1.000000
previous_loan_defaults_on_file            0.000000
loan_status                               0.000000

                                      previous_loan_defaults_on_file  \
person_age                                                 0.012727
person_gender                                              0.000763
person_education                                           0.001757
person_income                                             -0.023474
person_emp_exp                                             0.010356
person_home_ownership                                      0.000000
loan_amnt                                                  0.011656
loan_intent                                                0.000000
loan_int_rate                                              0.023202
loan_percent_income                                        0.048789
cb_person_cred_hist_length                                 0.011656
credit_score                                               0.000000
previous_loan_defaults_on_file                             1.000000
loan_status                                                0.000000

                                      loan_status
person_age                               0.010375
person_gender                            0.000763
person_education                         0.001757
person_income                           -0.544534
person_emp_exp                           0.010375
person_home_ownership                    0.000000
loan_amnt                                0.146755
loan_intent                              0.000000
loan_int_rate                            0.125687
loan_percent_income                      1.000000
cb_person_cred_hist_length               0.000000
credit_score                             0.000000
previous_loan_defaults_on_file           0.000000
loan_status                              1.000000
```

```
# Display correlation with target variable
print("Correlations with target (loan_status):")
print("Interpretation: High target variable (loan_status)")
Correlations with target (loan_status):
Interpretation: High target variable (loan_status)
```

```
# Sort by correlation strength (highest to lowest)
correlation_with_target = correlation[target].sort_values(ascending=False)
Correlation with target variable (loan_status):
loan_status                         1.000000
person_income                       0.544534
loan_percent_income                 0.544981
person_emp_exp                      0.010375
person_age                          0.010375
person_education                    0.001757
person_gender                       0.000763
previous_loan_defaults_on_file      0.011656
cb_person_cred_hist_length          0.000000
credit_score                        0.000000
person_home_ownership               0.000000
loan_intent                         0.000000
loan_amnt                           0.146755
loan_int_rate                       0.125687
Name: loan_status, dtype: float64
```

```
# Create a heatmap to visualize correlations
import matplotlib.pyplot as plt
sns.heatmap(correlation, annot=True, fmt='.2f', cmap='coolwarm', center=0,
            square=True, linewidths=0.5)
plt.title('Correlation Heatmap', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()
```

6. Normalize data using appropriate scaling

```
# Apply StandardScaler (for comparison)
# Create a StandardScaler object
scaler = StandardScaler()
# Fit the scaler to training data and transform it
X_scaled = scaler.fit_transform(X)
# Convert back to DataFrame to keep column names
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)
Applying StandardScaler using StandardScaler (based_data):
print("Scaled Data Statistics:")
print(X_scaled.describe()):
Data normalized using StandardScaler (based_data):
Scaled Data Statistics:
       person_age  person_gender  person_education  person_income  \
count  4.500000e+4   4.500000e+4       4.500000e+4   4.500000e+4
mean   1.111000e-16  -1.000711e-16     -1.811556e-17  -6.195755e-17
std    1.000011e+0   1.000011e+0       1.000011e+0   1.000011e+0
min    -1.239000e+0  -1.000711e+0      -1.811556e+0  -1.329753e+0
25%    -6.610000e-01  -1.000711e+0      -1.811556e+0  -6.195755e-01
50%    -2.910000e-01  -1.000000e+0      -1.811556e+0  -1.329753e-01
75%    2.310000e+0   9.993184e-01      9.993184e-01   5.346245e-01
max    1.010000e+01  9.993184e-01      9.993184e-01   1.011000e+01

       person_emp_exp  person_home_ownership  ...  loan_int_rate  \
count  4.500000e+4   4.500000e+4  ...  4.500000e+4
mean   1.111000e-16  -1.111000e-16  ...  -1.111000e-16
std    1.000011e+0   1.000011e+0  ...  1.000011e+0
min    -1.000000e+0  -1.000000e+0  ...  -1.860000e+0
25%    -6.610000e-01  -1.000000e+0  ...  -6.610000e-01
50%    -2.910000e-01  -1.000000e+0  ...  -2.910000e-01
75%    5.000000e-01   9.993184e-01  ...  5.000000e-01
max    1.010000e+01   9.993184e-01  ...  4.310000e+0

       loan_percent_income  cb_person_cred_hist_length  credit_score  \
count  4.500000e+4                4.500000e+4   4.500000e+4
mean   1.111000e-16               -1.111000e-16  -1.111000e-16
std    1.000011e+0                1.000011e+0   1.000011e+0
min    -1.580000e+0               -1.540000e+0  -3.100000e+0
25%    -7.100000e-01              -7.100000e-01  -6.100000e-01
50%    -1.100000e-01              -1.100000e-01  -1.100000e-01
75%    5.000000e-01               5.000000e-01   6.000000e-01
max    4.310000e+0                4.310000e+0   4.310000e+0

       previous_loan_defaults_on_file  loan_status
count                    4.500000e+4   4.500000e+4
mean                     -1.111000e-16  -1.111000e-16
std                      1.000011e+0   1.000011e+0
min                      -1.000000e+0  -1.000000e+0
25%                      -1.000000e+0  -1.000000e+0
50%                      -1.000000e+0  -1.000000e+0
75%                      9.993184e-01   9.993184e-01
max                      9.993184e-01   9.993184e-01

[8 rows x 13 columns]
```

7. Split the data of table accordingly

```
# Display data information
print("Training set class distribution:")
print(y_train.value_counts())
print("Test set class distribution:")
print(y_test.value_counts())
Training set class distribution:
loan_status
0    28000
1     8000
Name: count, dtype: int64

Testing set class distribution:
loan_status
0    7000
1    2000
Name: count, dtype: int64
```

```
# Create logistic regression model
model = LogisticRegression(random_state=42, max_iter=1000)
model.fit(X_train, y_train)
```

```
LogisticRegression
LogisticRegression(random_state=42, max_iter=1000)
```

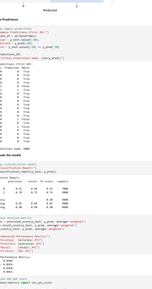
8. Fit a model

9. Create a confusion matrix to compare the predicted data activities to the actual activities and compute the accuracy.

```
# Make predictions on test set
# Use the trained model to make predictions on test data
y_pred = model.predict(X_test)  # Predicted classes (0 or 1)
y_pred_proba = model.predict_proba(X_test)[:, 1]  # Probability of approval
# Calculate accuracy, precision, recall, f1-score
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy: {accuracy:.4f}")
Model Accuracy: 0.8900
```

```
# Create confusion matrix
# This shows how many we got right vs wrong, and what type of errors
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
[[6572  428]
 [ 551 1449]]
Confusion Matrix:
[[6572  428]
 [ 551 1449]]
True Negatives: {cm[0,0]} (correctly denied)
False Positives: {cm[0,1]} (incorrectly approved)
False Negatives: {cm[1,0]} (incorrectly denied)
True Positives: {cm[1,1]} (correctly approved)
```

```
# Visualize confusion matrix
import matplotlib.pyplot as plt
import seaborn as sns
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```



10. Make Predictions

```
# Display sample predictions
print("Sample Predictions (first 20):")
print("Actual | Predicted | Match")
for i in range(20):
    actual = y_test.iloc[i]
    predicted = y_pred[i]
    match = "✓" if actual == predicted else "✗"
    print(f"{actual:6} | {predicted:9} | {match}")
Sample Predictions (first 20):
Actual | Predicted | Match
0      | 0         | ✓
0      | 0         | ✓
0      | 0         | ✓
0      | 0         | ✓
0      | 0         | ✓
0      | 0         | ✓
0      | 0         | ✓
0      | 0         | ✓
0      | 0         | ✓
0      | 0         | ✓
0      | 0         | ✓
0      | 0         | ✓
0      | 0         | ✓
0      | 0         | ✓
0      | 0         | ✓
0      | 0         | ✓
0      | 0         | ✓
0      | 0         | ✓
0      | 0         | ✓
0      | 0         | ✓
Total predictions made: 9000
```

11. Evaluate the model

```
# Display classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))
Classification Report:
              precision    recall  f1-score   support

           0       0.92      0.94      0.93      7000
           1       0.77      0.72      0.74      2000

    accuracy                           0.90      9000
   macro avg       0.85      0.83      0.84      9000
weighted avg       0.89      0.90      0.89      9000
```

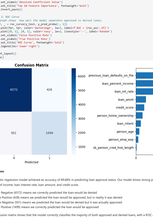
```
# Calculate detailed metrics
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')
print("Detailed Performance Metrics:")
print("Accuracy: {accuracy:.4f}")
print("Precision: {precision:.4f}")
print("Recall: {recall:.4f}")
print("F1-Score: {f1:.4f}")
Detailed Performance Metrics:
Accuracy: 0.8900
Precision: 0.8941
Recall: 0.8900
F1-Score: 0.8921
```

```
# Calculate ROC AUC score
from sklearn.metrics import roc_auc_score
roc_auc = roc_auc_score(y_test, y_pred_proba)
print("ROC AUC Score: {roc_auc:.4f}")
ROC AUC Score: 0.9314
```

```
# Analyze feature importance (coefficients)
feature_importance = pd.DataFrame(
    'Feature': X.columns,
    'Coefficient': model.coef_[0]
)
feature_importance['Abs_Coefficient'] = abs(feature_importance['Coefficient'])
feature_importance = feature_importance.sort_values('Abs_Coefficient', ascending=False)
print("Top 5 Most Important Features:")
print(feature_importance[['Feature', 'Coefficient']].head())
Top 5 Most Important Features:
                            Feature  Coefficient
10  previous_loan_defaults_on_file    -3.324010
9              loan_percent_income    2.750000
3                    person_income    -1.329753
4                    person_emp_exp    0.993184
0                       person_age    0.660100
```

12. Generate relevant graphs

```
# Imports for visualization
import matplotlib.pyplot as plt, seaborn as sns
# Create figure with multiple subplots
fig, axes = plt.subplots(2, 2, figsize=(15, 12))
# Plot 1: Confusion Matrix
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['Predicted 0', 'Predicted 1'],
            yticklabels=['Actual 0', 'Actual 1'], ax=axes[0,0])
axes[0,0].set_title('Confusion Matrix')
# Plot 2: Feature Importance
top_features = feature_importance.head(10)
colors = ['red' if x < 0 else 'blue' for x in top_features['Coefficient']]
axes[0,1].barh(range(len(top_features)), top_features['Abs_Coefficient'])
axes[0,1].set_yticks(range(len(top_features)))
axes[0,1].set_yticklabels(top_features['Feature'])
axes[0,1].set_xlabel('Absolute Coefficient Value')
axes[0,1].set_title('Top 10 Feature Importance')
axes[0,1].invert_yticks()
# Plot 3: ROC Curve
from sklearn.metrics import roc_curve
fpr, tpr, _ = roc_curve(y_test, y_pred_proba)
axes[1,0].plot(fpr, tpr, 'b-', linewidth=2, label=f'ROC (AUC = {roc_auc:.3f})')
axes[1,0].plot([0, 1], [0, 1], 'r--', linewidth=1, label='Random')
axes[1,0].set_xlabel('False Positive Rate')
axes[1,0].set_ylabel('True Positive Rate')
axes[1,0].set_title('ROC Curve')
axes[1,0].legend(loc='lower right')
plt.tight_layout()
plt.show()
```



Conclusion

The logistic regression model achieved an accuracy of 89.89% in predicting loan approval status. Our model shows strong performance with a high precision of 89.54% and a recall of 89.89%. The most influential features for prediction are previous loan defaults on file, loan percent of income, loan interest rate, loan amount, and credit score.

The confusion matrix shows that the model correctly classifies the majority of both approved and denied loans, with a ROC AUC score of 0.93. Generally, the closer the ROC score is to 1, the better the model.

- True Negatives (6572) means we correctly predicted that loans should be denied
- False Positives (428) means we predicted the loan would be approved, but in reality it was denied
- False Negatives (551) means we incorrectly predicted loans should be denied, but they were approved
- True Positives (1449) means we correctly predicted the loan would be approved